Lista de exercícios – OO: Modelagem e Relacionamento

Orientações para resolução da lista

- Esta lista trabalha exclusivamente o conteúdo de **modelagem e relacionamento entre objetos**: identificação de classes, atributos e métodos a partir de um enunciado e as relações **usa** (o objeto é recebido como parâmetro), **tem um** (o objeto é guardado em um atributo) e **tem vários** (uma lista de objetos é guardada em um atributo).
- Todas as questões são de **construção de código**. Antes de implementar, desenhe o diagrama das classes e indique as relações entre elas com setas. Lembre-se: substantivos viram classes e atributos; verbos viram métodos.
- Cada objeto deve cuidar dos próprios dados. Quando um objeto precisa de uma informação de outro, ele acessa o atributo ou chama um método desse outro objeto, como em `self.arma.usar()` ou `item.valor`.
- Guardar um objeto em um atributo ou em uma lista não cria uma cópia: as duas variáveis passam a apontar para o **mesmo** objeto. Alterações feitas por uma delas aparecem na outra.
- Uma lista que é atributo de um objeto deve ser criada **dentro do `__init__`**, com `self.itens = []`, para que cada objeto tenha a sua própria lista.
- Em cada questão, escreva um programa principal que crie os objetos, relacione-os e chame os métodos, reproduzindo os exemplos do enunciado.
- Desenvolva as soluções individualmente. O objetivo da lista é consolidar seu aprendizado e desenvolver autonomia na construção de programas.

Exercícios

1. Considere a seguinte descrição de um sistema de academia:

"A academia possui alunos e instrutores. Cada aluno tem um plano, que define o valor da mensalidade e a quantidade de aulas por semana. Os alunos podem se matricular em treinos, e cada treino é montado por um instrutor e possui vários exercícios. O instrutor pode montar e alterar treinos. Os alunos podem pagar a mensalidade e registrar a presença."

Identifique as classes do sistema e, para cada uma, indique pelo menos 2 atributos e 2 métodos. Em seguida, indique todas as relações entre as classes, classificando cada uma como *usa*, *tem um* ou *tem vários*.

Por fim, implemente em Python as classes `Plano` e `Aluno`, de forma que um aluno **tenha um** plano e possua o método `exibir()`, que mostra o nome do aluno, o valor da mensalidade e a quantidade de aulas por semana do seu plano.

2. Crie as classes `Paciente` (nome, idade e saldo de créditos) e `Consulta` (especialidade e valor).

Implemente na classe `Paciente` o método `agendar(consulta)`, que recebe um objeto `Consulta` (relação *usa*). Se o paciente tiver créditos suficientes, o valor da consulta deve ser descontado do saldo e uma confirmação deve ser exibida. Caso contrário, uma mensagem de recusa deve ser exibida.

Exemplo de uso:

```
paciente = Paciente("Joana", 34, 300)
c1 = Consulta("Cardiologia", 250)
c2 = Consulta("Dermatologia", 180)
paciente.agendar(c1)
paciente.agendar(c2)
print(paciente.saldo)
```

Saída esperada:

```
Joana agendou Cardiologia (R$ 250.00)
Joana nao tem creditos para Dermatologia
50
```

3. Crie as classes `Motor` e `Carro`. Um motor tem potência (em cavalos) e um atributo que indica se está ligado. Ele possui os métodos `ligar()` e `desligar()`.

Um carro tem modelo, velocidade (inicialmente 0) e **tem um** motor, recebido pelo construtor. A classe `Carro` deve ter os métodos:

- `dar_partida()`: pede ao motor que ligue;
- `acelerar(valor)`: aumenta a velocidade, mas apenas se o motor estiver ligado. A velocidade máxima do carro é igual à potência do motor; a velocidade nunca pode ultrapassá-la;
- `frear()`: zera a velocidade;
- `desligar()`: só desliga o motor se o carro estiver parado.

Exemplo de uso:

```
motor = Motor(120)
carro = Carro("Sedan", motor)
carro.acelerar(50)
carro.dar_partida()
carro.acelerar(80)
carro.acelerar(80)
carro.desligar()
print(carro.velocidade, motor.ligado)
```

Saída esperada:

```
Ligue o motor primeiro
Pare o carro antes de desligar
120 True
```

Observe que `motor` e `carro.motor` são o mesmo objeto.

4. Crie as classes `Musica`, `Usuario` e `Playlist`. Uma música tem título, artista e duração em segundos. Um usuário tem nome e e-mail. Uma playlist **tem um** dono (um objeto `Usuario`), tem um nome e **tem várias** músicas.

A classe `Playlist` deve ter os métodos:

- `adicionar_musica(musica)`: acrescenta a música, desde que ela ainda não esteja na playlist (compare pelo título);
- `duracao_total()`: devolve a soma das durações;
- `exibir_playlist()`: mostra o nome da playlist, o nome do dono, cada música no formato `titulo - artista (m:ss)` e a duração total no mesmo formato.

Exemplo de uma música de 215 segundos: `Aurora - Banda Sol (3:35)`.

5. Crie as classes `Item`, `Inventario` e `Personagem`. Um item tem nome, valor e peso. Um inventário tem uma capacidade máxima de peso e **tem vários** itens. Um personagem tem nome e **tem um** inventário, que deve ser criado **dentro do** `__init__` do personagem.

A classe `Inventario` deve ter os métodos:

- `peso_atual()`: devolve a soma dos pesos dos itens guardados;
- `adicionar_item(item)`: guarda o item apenas se ele couber na capacidade restante, devolvendo `True` ou `False`;
- `valor_total()`: devolve a soma dos valores;
- `exibir_itens()`: mostra cada item e, ao final, o peso ocupado e a capacidade.

O método `coletar(item)` do personagem (relação *usa*) deve repassar o item ao inventário e exibir se a coleta foi feita ou se o inventário está cheio.

No programa principal, crie um personagem com capacidade 10 e faça-o coletar uma Espada (valor 50, peso 6), uma Pocao (valor 25, peso 1) e um Escudo (valor 40, peso 5). O escudo não deve caber. Exiba o inventário e o valor total. Por fim, mostre o nome do primeiro item usando apenas a variável do personagem: `heroi.inventario.itens[0].nome`.

6. Crie as classes `Cliente`, `Produto` e `Pedido`. Um cliente tem nome e cidade. Um produto tem nome e preço. Um pedido tem um número, **tem um** cliente e **tem vários** produtos, cada um acompanhado da quantidade pedida.

A classe `Pedido` deve ter os métodos:

- `adicionar_produto(produto, quantidade)`: se o produto já estiver no pedido, soma a quantidade; caso contrário, inclui o produto com a quantidade informada;
- `calcular_total()`: devolve o valor total do pedido;
- `calcular_frete()`: devolve 0 se o total for maior ou igual a 200; caso contrário, devolve 15 se o cliente for de `Belo Horizonte` e 30 nas demais cidades;
- `exibir_resumo()`: mostra o número do pedido, o nome do cliente, cada produto com quantidade e subtotal, o total, o frete e o valor final.

Para guardar cada produto com sua quantidade, você pode usar uma lista de pares `[produto, quantidade]`. Atenção: o preço de cada produto deve ser lido do objeto `Produto` no momento do cálculo, e não copiado para o pedido.

No programa principal, crie dois pedidos para o mesmo cliente com produtos em comum. Em seguida, altere o preço de um produto (por exemplo, `produto.preco = 80`) e exiba novamente os dois resumos. Observe e explique, em um comentário no código, por que os dois pedidos foram afetados.

7. Crie as classes `Inimigo` e `Fase`. Um inimigo tem nome, vida e dano. Uma fase tem nome e **tem vários** inimigos.

A classe `Inimigo` deve ter os métodos `receber_dano(valor)`, que reduz a vida sem deixá-la negativa, `esta_vivo()` e `exibir()`.

A classe `Fase` deve ter os métodos:

- `adicionar_inimigo(inimigo)`;
- `ataque_em_area(valor)`: causa o mesmo dano em todos os inimigos vivos da fase;
- `inimigos_vivos()`: devolve uma **nova lista** contendo apenas os inimigos ainda vivos;
- `mais_perigoso()`: devolve o objeto inimigo vivo com o maior dano, ou `None` se não houver inimigos vivos;
- `concluida()`: devolve `True` quando nenhum inimigo estiver vivo.

No programa principal, crie uma fase com Goblin (vida 10, dano 3), Morcego (vida 5, dano 2) e Orc (vida 20, dano 7). Aplique ataques em área de 6 pontos até a fase ser concluída e, após cada ataque, exiba os inimigos vivos e o nome do mais perigoso. Ao final, informe quantos ataques foram necessários.

8. Uma biblioteca deseja controlar seus empréstimos. Crie as classes `Livro` (título, autor e disponível), `Leitor` (nome e lista de livros emprestados) e `Biblioteca` (nome e lista de livros do acervo).

Implemente os seguintes métodos:

- na `Biblioteca`: `adicionar_livro(livro)`, `buscar(titulo)`, que devolve o objeto `Livro` com aquele título ou `None`, e `exibir_disponiveis()`;
- no `Leitor`: `pegar(livro)` e `devolver(livro)`. Um leitor pode ter no máximo 2 livros ao mesmo tempo. Ao pegar um livro, ele deve ficar indisponível; ao devolver, deve voltar a ficar disponível. Um livro indisponível não pode ser pego, e um leitor não pode devolver um livro que não está com ele.

No programa principal, crie uma biblioteca com quatro livros e dois leitores. Simule uma sequência de empréstimos e devoluções, buscando os livros pelo título, incluindo tentativas que devem ser recusadas, e exiba os livros disponíveis ao final.

Observe que o mesmo objeto `Livro` fica, ao mesmo tempo, na lista da biblioteca e na lista do leitor: ao marcá-lo como indisponível pelo leitor, a biblioteca enxerga a mudança.

9. Crie as classes `Jogador` (nome e pontos), `Partida` (número da rodada, **tem um** jogador vencedor e pontos em disputa) e `Campeonato` (nome, **tem vários** jogadores e **tem várias** partidas).

A classe `Campeonato` deve ter os métodos:

- `inscrever(jogador)`: inscreve o jogador, desde que não exista outro com o mesmo nome;
- `registrar_partida(nome_vencedor, pontos)`: localiza o jogador pelo nome, cria um objeto `Partida` com o número da próxima rodada, guarda essa partida e soma os pontos ao jogador. Se o jogador não estiver inscrito, a partida não deve ser registrada;
- `vitorias(jogador)`: devolve quantas partidas aquele jogador venceu;
- `exibir_classificacao()`: mostra os jogadores em ordem decrescente de pontos, com a quantidade de vitórias de cada um.

Para montar a classificação, construa uma nova lista selecionando, a cada passo, o jogador com mais pontos que ainda não foi escolhido. Não utilize `sorted()` nem `sort()`.

10. Um sistema de hotel precisa controlar quartos e reservas. Crie as classes:

- `Hospede`: nome, CPF e um percentual de desconto (por exemplo, 10 para 10%);
- `Quarto`: número, valor da diária e capacidade de pessoas;
- `Reserva`: código, **tem um** hóspede, **tem um** quarto, data de entrada e quantidade de diárias;
- `Hotel`: nome, **tem vários** quartos e **tem várias** reservas.

A classe `Reserva` deve ter o método `calcular_total()`, que usa a diária do quarto e o desconto do hóspede, e o método `exibir()`.

A classe `Hotel` deve ter os métodos:

- `adicionar_quarto(quarto)`;
- `quarto_ocupado(numero, data)`: devolve `True` se já existir uma reserva para aquele quarto naquela data de entrada;
- `reservar(hospede, numero, data, diarias, pessoas)`: cria e guarda a reserva apenas se o quarto existir, se a capacidade comportar a quantidade de pessoas e se o quarto não estiver ocupado naquela data. O código da reserva deve ser gerado automaticamente (1, 2, 3, ...). O método deve devolver o objeto `Reserva` criado ou `None`, exibindo o motivo da recusa;
- `reservas_do_hospede(cpf)`: devolve a lista de reservas daquele hóspede;
- `faturamento()`: devolve a soma dos totais de todas as reservas.

Para simplificar, considere que as datas são textos como `"10/12"` e que duas reservas só conflitam quando têm a mesma data de entrada.

No programa principal, crie um hotel com três quartos e dois hóspedes, tente realizar pelo menos cinco reservas (algumas devem ser recusadas, cada uma por um motivo diferente), exiba as reservas de um dos hóspedes e o faturamento do hotel.